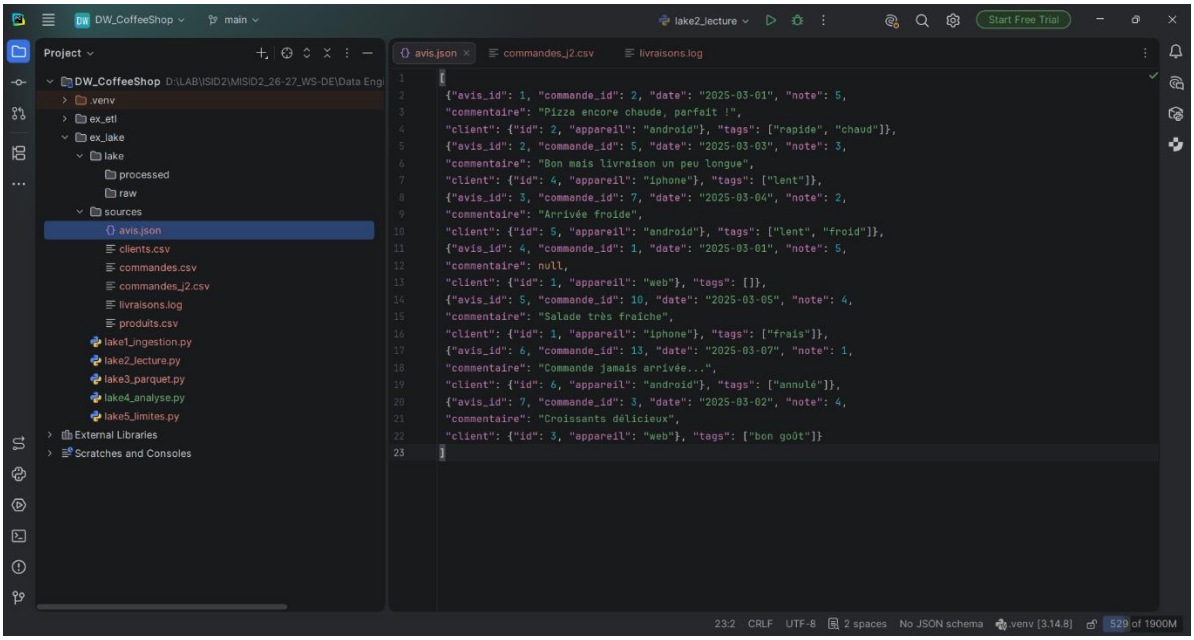
31/10/2026

Compte rendu de TP

Construire un Data Lake

Zied.Brahmi
MISID2

Étape 0 : Préparation

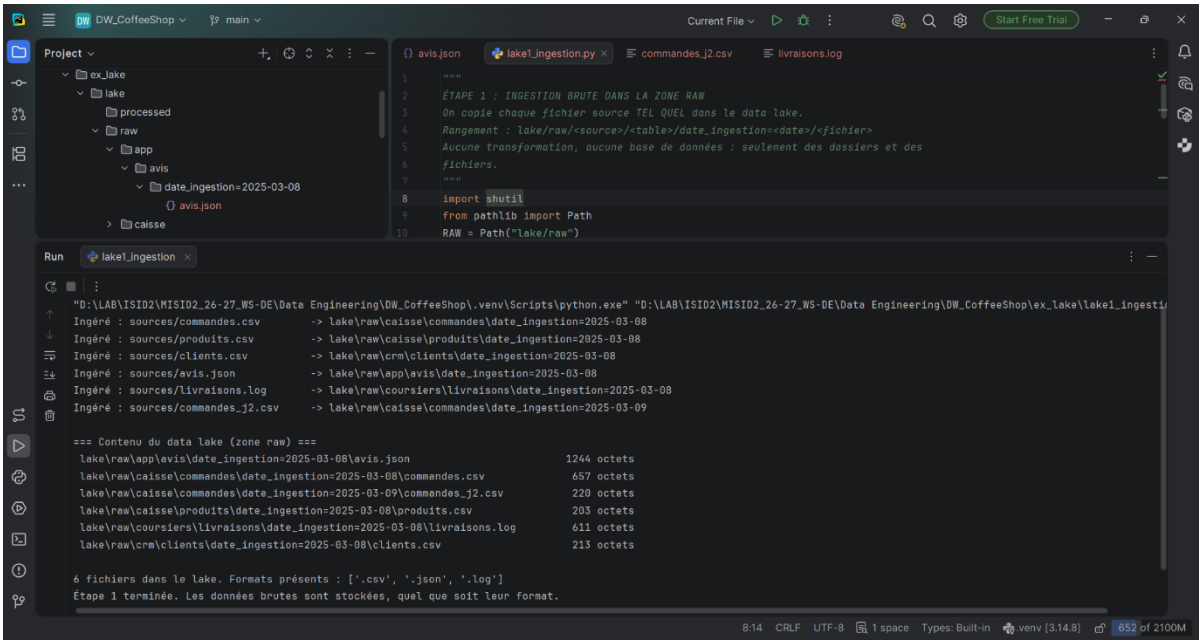


Questions :

1. Structure des données sources :

Source	Type	Chargement directe
avis.json	Semi-structuré	Non
clients.csv	Structuré	Oui
commandes.csv	Structuré	Oui
commandes_j2.csv	Structuré	Oui
livraisons.log	Non structuré	Non
produits.csv	Structuré	Oui

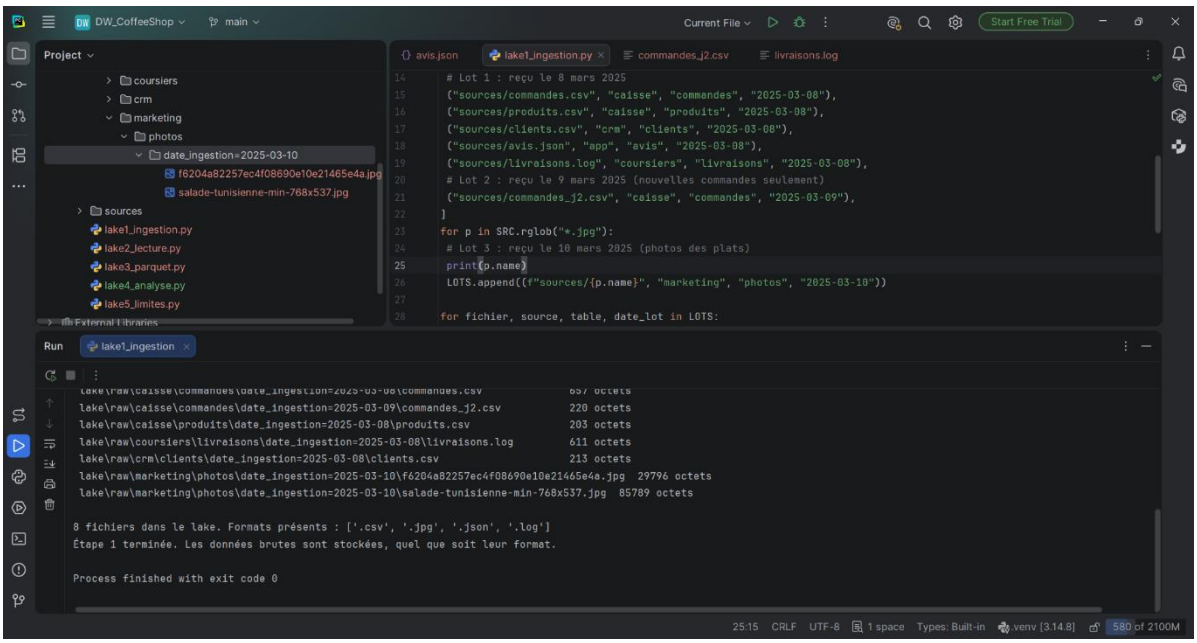
Étape 1 : Ingestion brute dans la zone raw



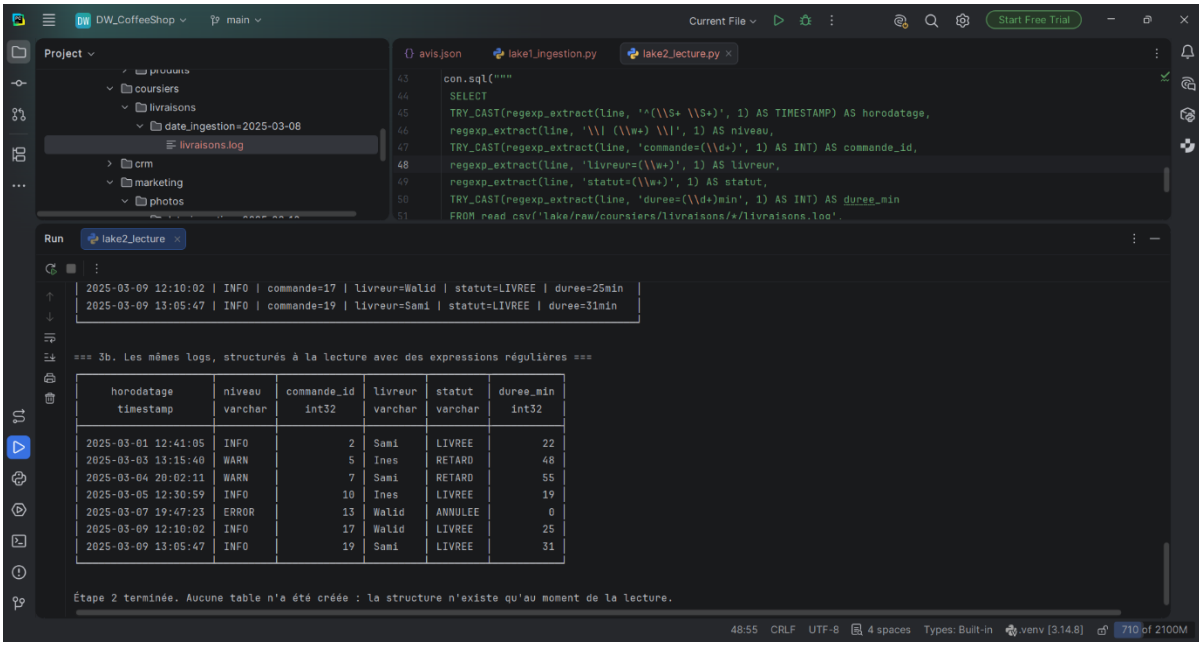
Questions :

1. On range les fichiers par date d'ingestion plutôt que d'écraser les anciens fichiers pour garder toujours une copie originale (sans modifications) des fichiers sources afin de refaire l'étape transformation en cas de besoin.
2. Les doublons des commandes ne sont pas corrigés parce que dans cette étape on garde les données sources intactes.
3. En lançant le script une deuxième fois, le nombre de fichiers (six) ne change pas parce le script utilise les même timestamp ce qui va écraser les anciens fichiers.
4. Pour ingérer les photos des plats, il faudrait ajouter au script le code nécessaire pour ramener toutes les images du répertoire source vers la zone raw du lake, par example comme suit :

```
RAW = Path("lake/raw")
SRC = Path("sources")
# (fichier source, système source, table, date du lot d'ingestion)
LOTS = [
 # Lot 1 : reçu le 8 mars 2025
 ("sources/commandes.csv", "caisse", "commandes", "2025-03-08"),
 ("sources/produits.csv", "caisse", "produits", "2025-03-08"),
 ("sources/clients.csv", "crm", "clients", "2025-03-08"),
 ("sources/avis.json", "app", "avis", "2025-03-08"),
 ("sources/livraisons.log", "coursiers", "livraisons", "2025-03-08"),
 # Lot 2 : reçu le 9 mars 2025 (nouvelles commandes seulement)
 ("sources/commandes_j2.csv", "caisse", "commandes", "2025-03-09"),
]
for p in SRC.rglob("*.jpg"):
 # Lot 3 : reçu le 10 mars 2025 (photos des plats)
 print(p.name)
 LOTS.append((f"sources/{p.name}", "marketing", "photos", "2025-03-
10"))
```



Étape 2 : schema-on-read, lire chaque format en SQL



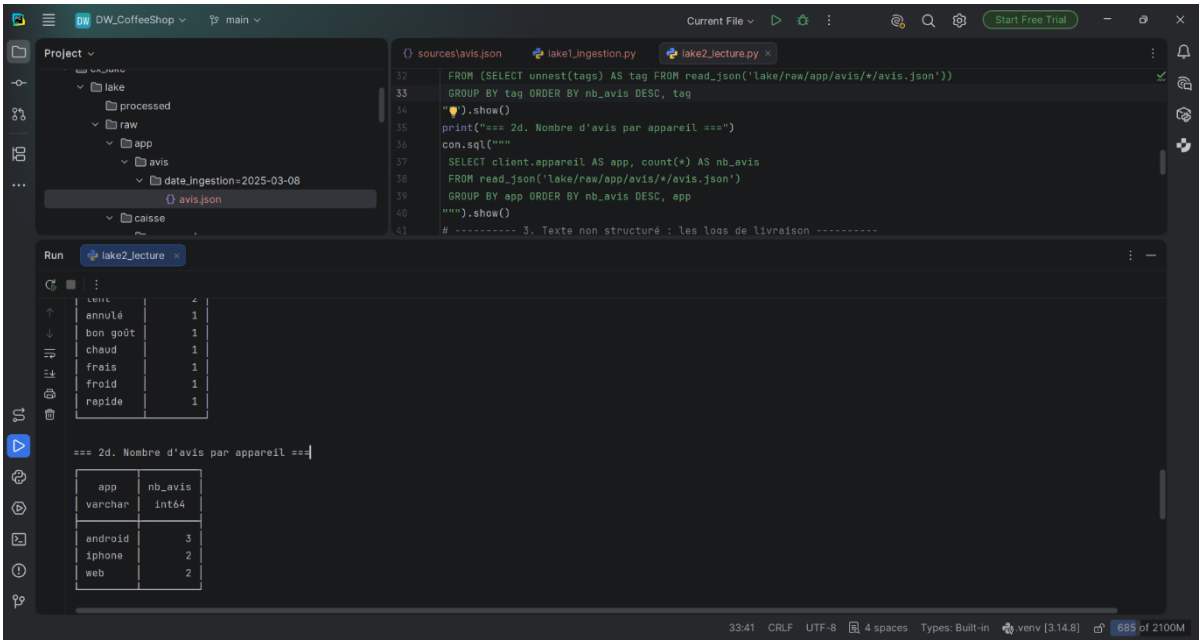
Questions :

1. Les « tables » que l'on vient d'interroger sont volatiles : se trouvent seulement en mémoire.
2. *date_commande* est devinée comme du texte (VARCHAR) et non comme une date parce certaines dates sont à la forme française (non ISO).
3. A la partie 3b si un coursier écrivait *duree=environ 20min* deux cas se présentent :
   - Avec un cast directe (::INT ou CAST *duree* AS INT) le script tombe en erreur et quitte

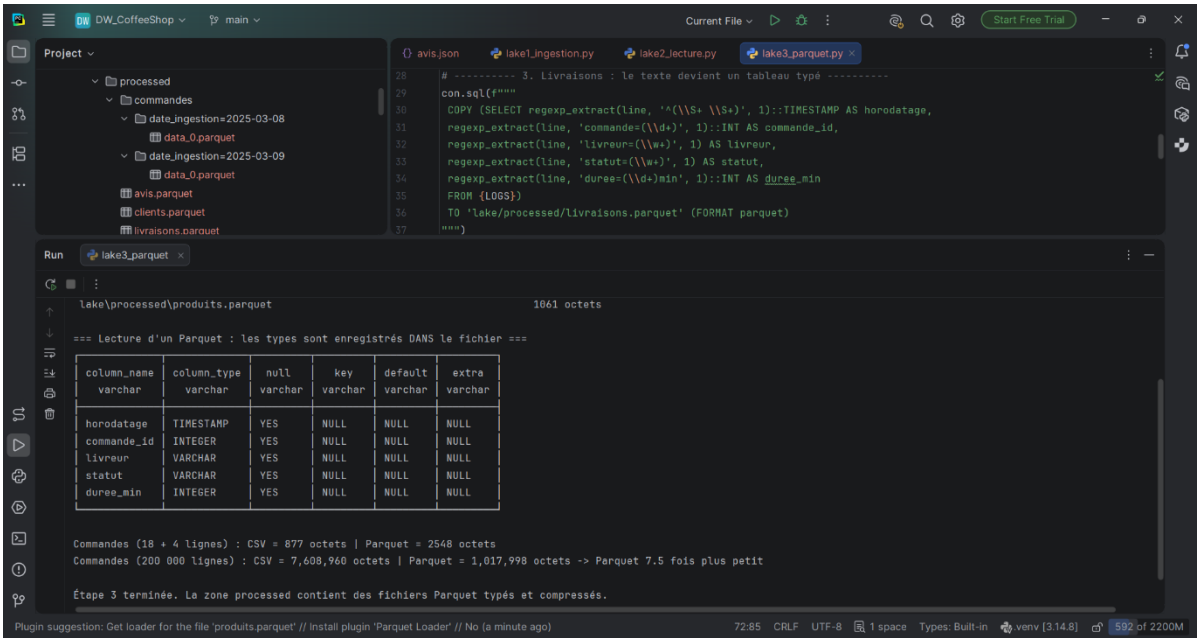
- Avec un TRY_CAST *duree* AS INT le script retourne une valeur NULL pour *duree*

4. Avis par appareil :

```
print("=== 2d. Nombre d'avis par appareil ===")
con.sql("""
 SELECT client.appareil AS app, count(*) AS nb_avis
 FROM read_json('lake/raw/app/avis/*/avis.json')
 GROUP BY app ORDER BY nb_avis DESC, app
""").show()
```



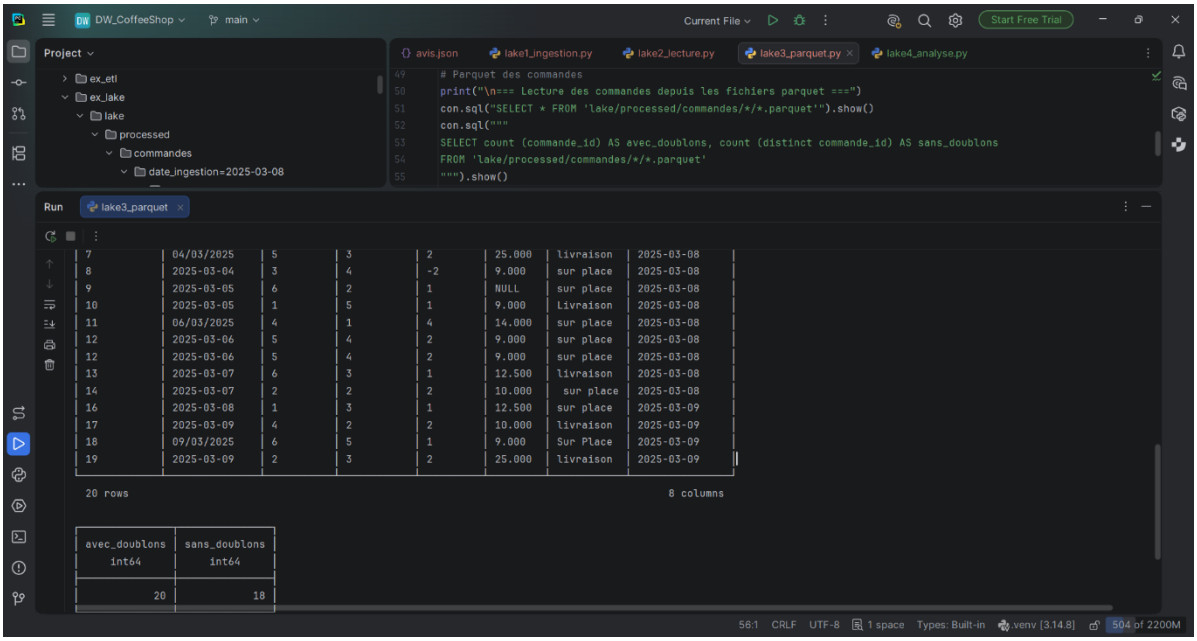
Étape 3 : zone processed en Parquet



Questions :

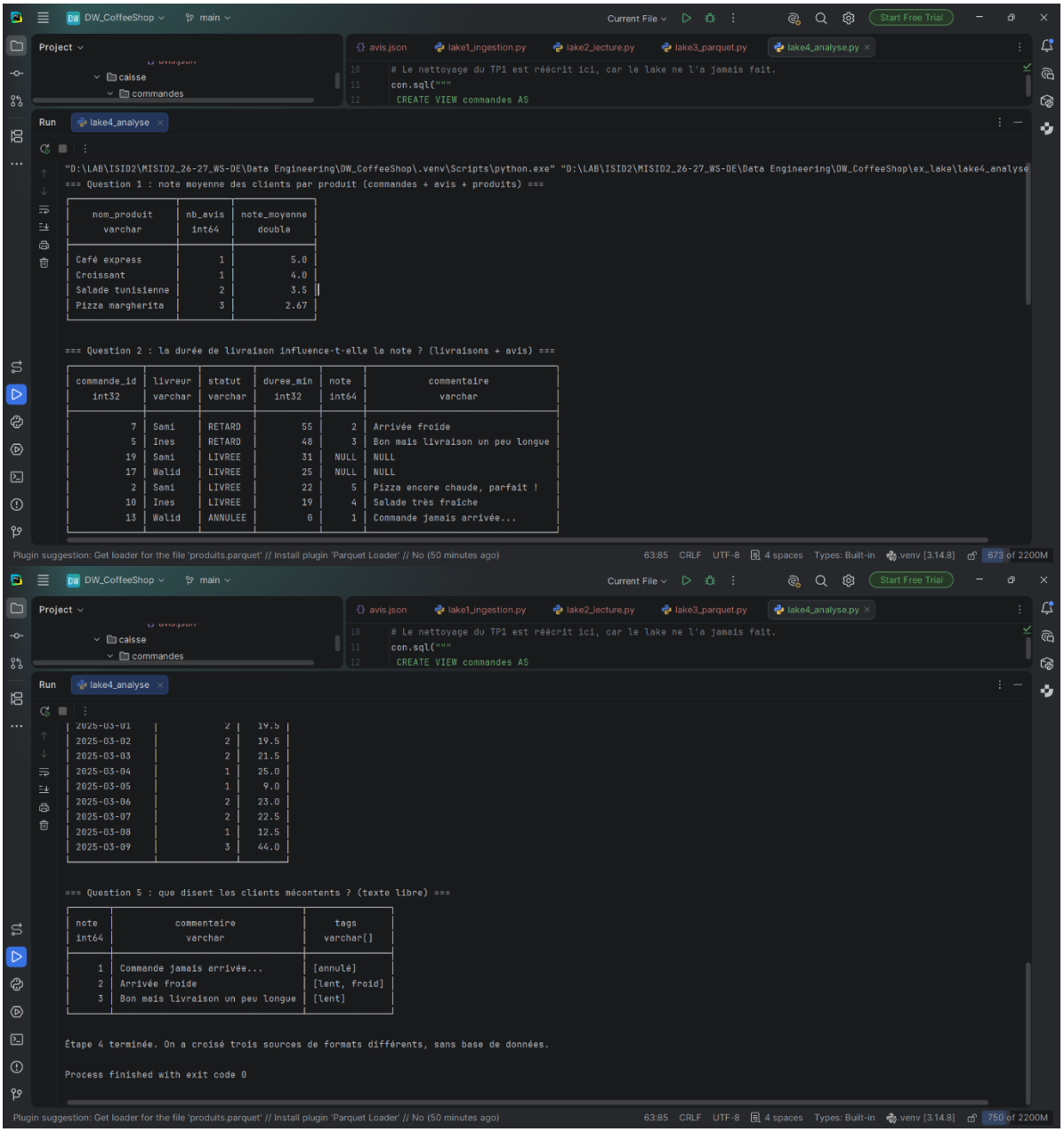
1. lake/processed/avis.parquet ne peut pas être lu comme un fichier texte parce ce qu'il est binaire.
2. Les doublons des commandes 6, 12 et 16 sont toujours présents.

```
# Parquet des commandes
print("\n=== Lecture des commandes depuis les fichiers parquet ===")
con.sql("SELECT * FROM
'lake/processed/commandes/*/*.parquet'").show()
con.sql("""
SELECT count (commande_id) AS avec_doublons, count (distinct
commande_id) AS sans_doublons
FROM 'lake/processed/commandes/*/*.parquet'
""").show()
```



3. Le parquet est plus gros que le csv pour les petites taille parce c'est un fichier binaire avec des métadonnées de taille importante par rapport aux fichiers plats.
4. Par rapport à row nous avons modifié la structure des fichiers dans processed.

Étape 4 : Analyses croisées directement sur le lake



Questions :

1. Le gérant peut déduire de la question 2 que le délai de livraison est un facteur très important pour la satisfaction du client.
2. Résultats après suppression DISTINCT de la vue *commandes* :

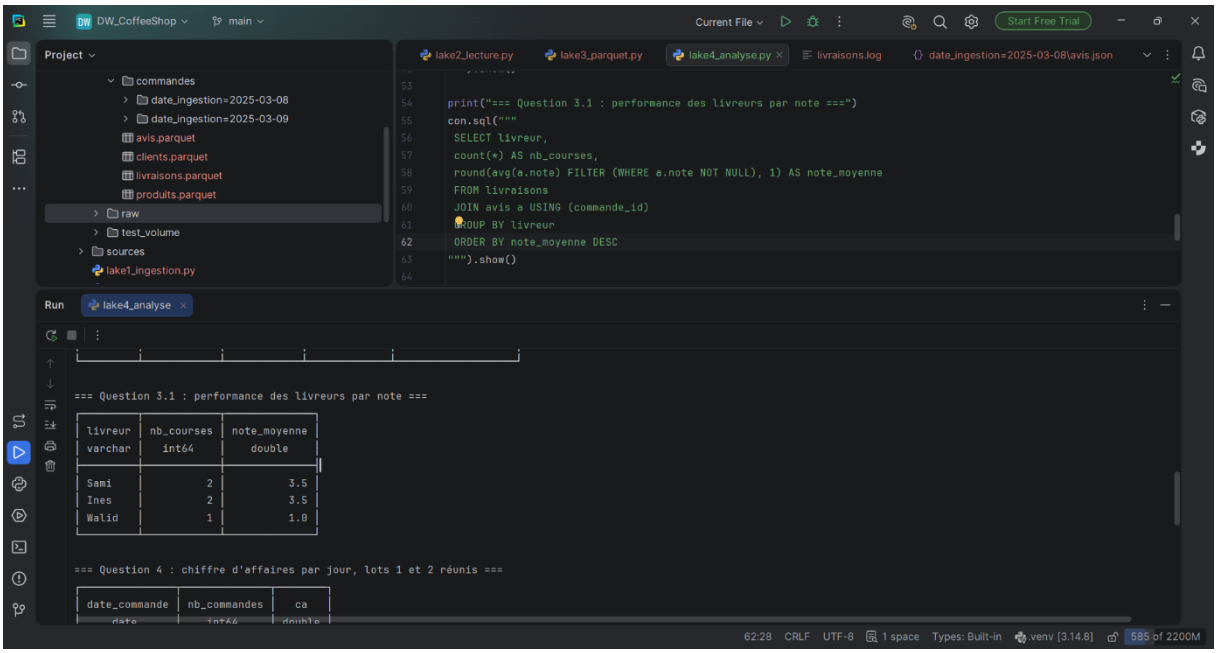
date_commande	nb_cmd avant	nb_cmd après	CA avant	CA après	Explication
2025-03-03	2	3	21.5	25.0	Doublon (6)
2025-03-06	2	3	23.0	32.0	Doublon (12)

2025-03-08	1	1	12.5	12.5	Pas de doublons pour ce jour

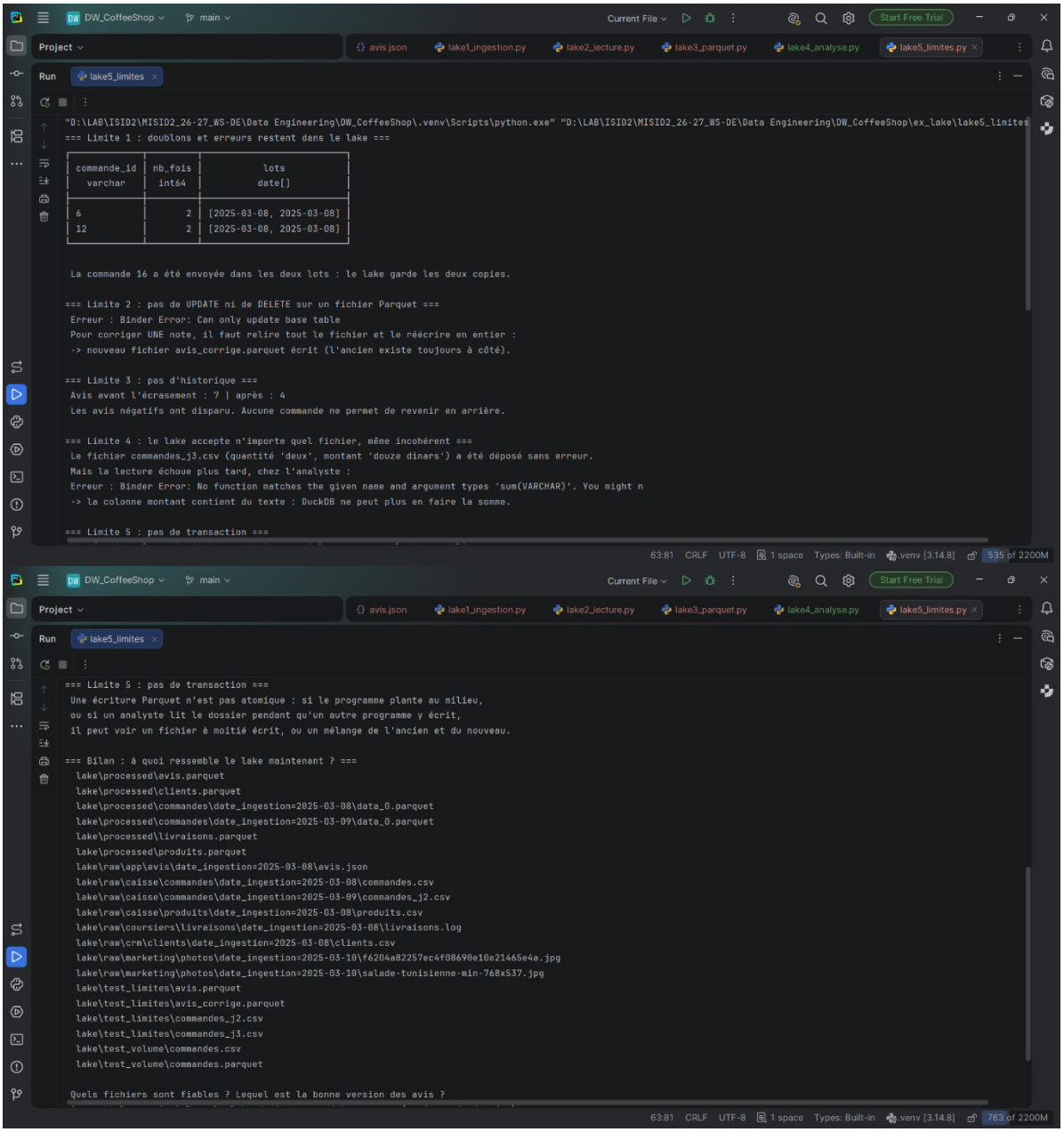
3. Pour faire l'analyse CA par canal, le deuxième analyste doit créer sa propre vue avec risques d'erreurs et omissions ce qui mène à des résultats différents du premier analyste.

4. Note moyenne par livreur :

```
print("=== Question 3.1 : performance des livreurs par note ===")
con.sql("""
 SELECT livreur,
 count(*) AS nb_courses,
 round(avg(a.note) FILTER (WHERE a.note NOT NULL), 1) AS note_moyenne
 FROM livraisons
 JOIN avis a USING (commande_id)
 GROUP BY livreur
 ORDER BY note_moyenne DESC
""").show()
```



Étape 5 : Les limites du data lake



Questions :

1. Impact des limites :

Limite	Example de conséquence
1. Pas de qualité garantie	Un prix avec virgule (au lieu de point) dans une commande fausse le CA
2. Pas de modification de ligne	Pour corriger une quantité dans une commande il faut réécrire tout le fichier commandes
3. Pas d'historique	Si par erreur le fichier avis.json a été modifié il n'y a plus de retour vers l'arrière
4. Pas de contrôle du schéma	N'importe quel fichier commande mal former est accepté
5. Pas de transaction	Lors d'une concurrence entre écriture et lecture du fichier log des livreurs on peut récupérer des données incorrectes

2. Dans le data warehouse du TP1, le fichier commandes_j3.csv serait rejeté à l'étape 3 (transformation)
3. Oui, la zone raw est concerné par la limite pas d'historique et elle peut être protégée en interdisant les accès en écriture.

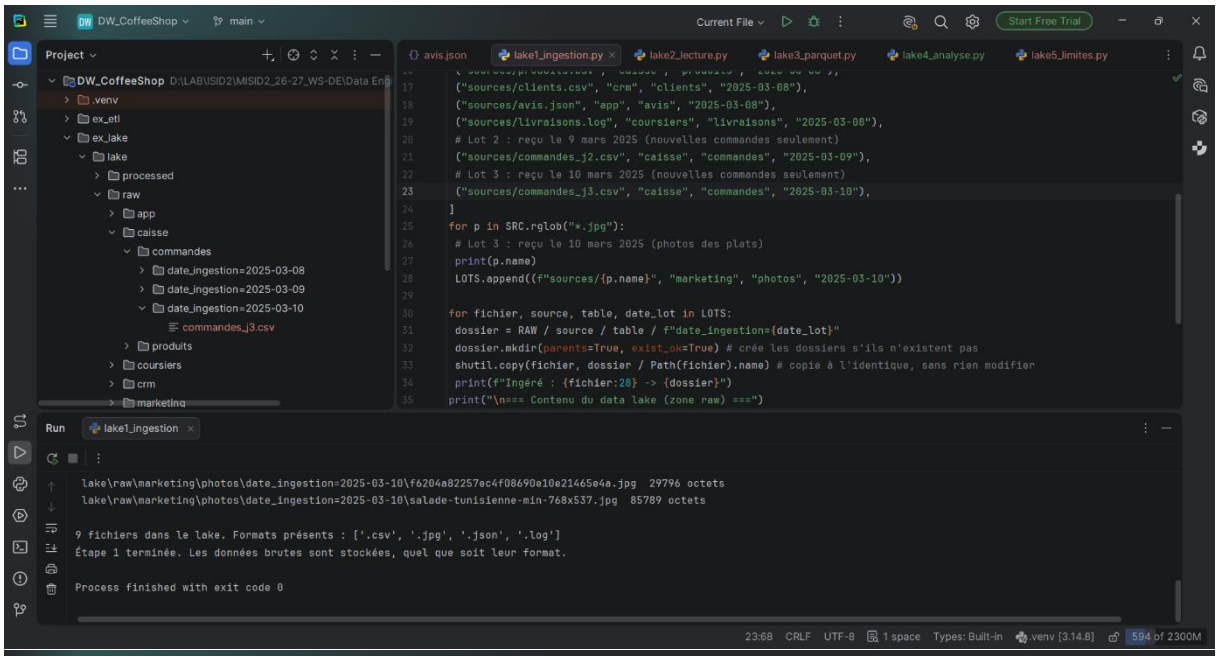
4. Pour éviter que ce lake devienne un data swamp on propose de :
   - Créer un catalogue de données
   - Définir un cycle de vie pour ces données

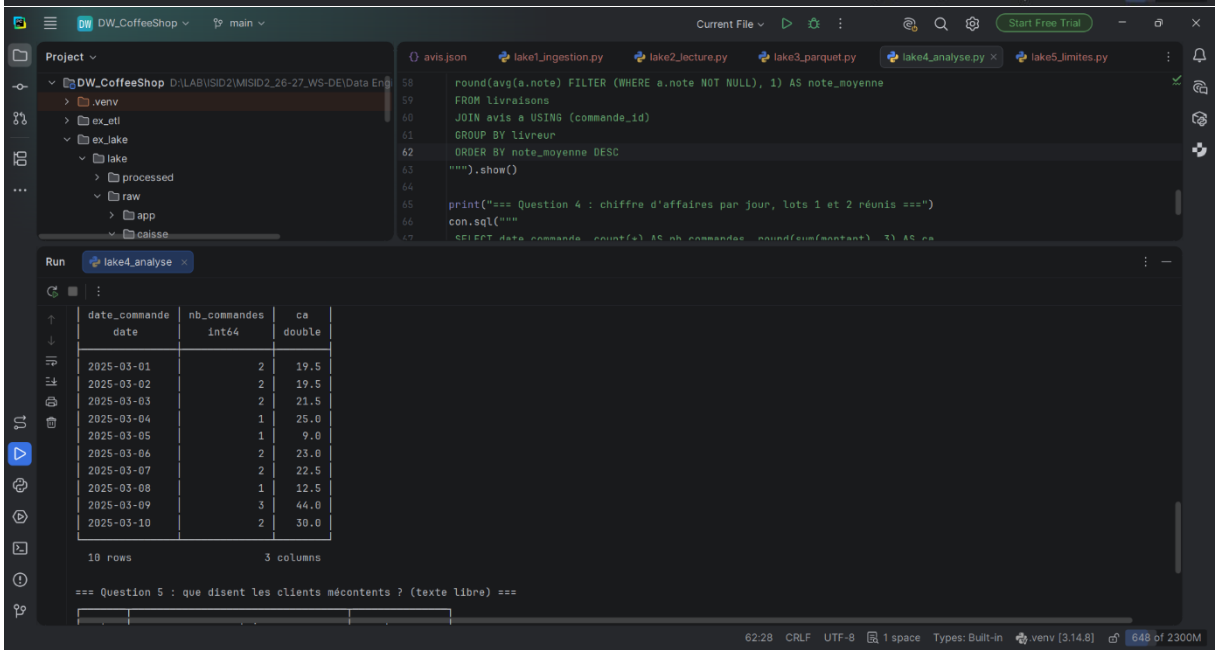
Bilan : data warehouse (TP1) ou data lake (TP2) ?

Critère	Data WH	Data lake
Peut stocker des logs et du JSON imbriqué tels quels	Non	Oui
Les données sont nettoyées une fois pour toutes	Oui	Non
La structure est définie à l'écriture ou à la lecture	En écriture	En lecture
Un fichier mal formé est refusé dès son arrivée	Oui	Non
On peut modifier une seule ligne	Oui	Non
On peut revenir à une version passée	Oui	Non
Coût de stockage	Plus cher	Moin cher
Utilisateurs les mieux servis	BI Analysts Décideurs et Managers	Data Scientists Data Engineers

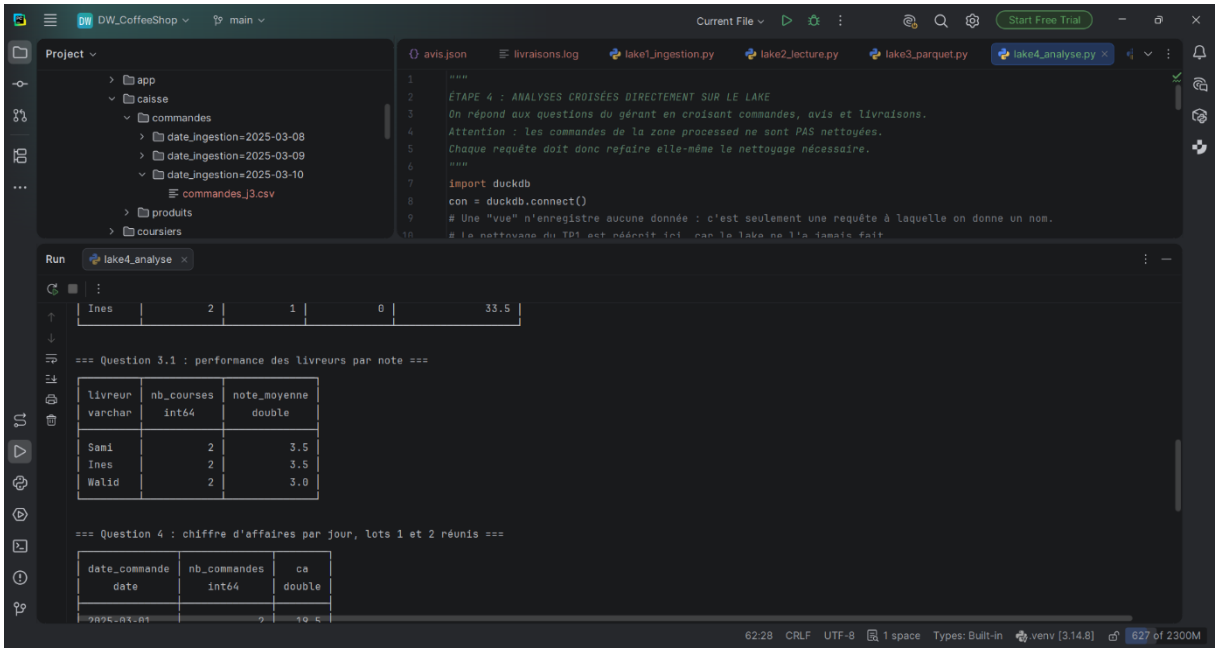
Exercices de manipulation

1. Un troisième lot de commandes :

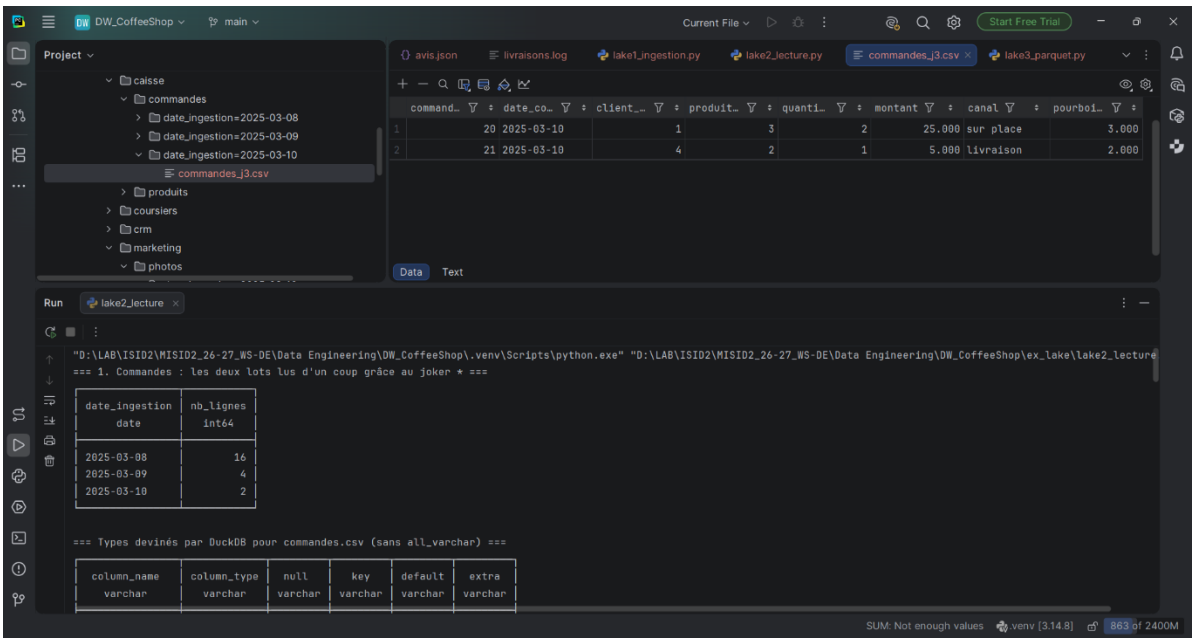




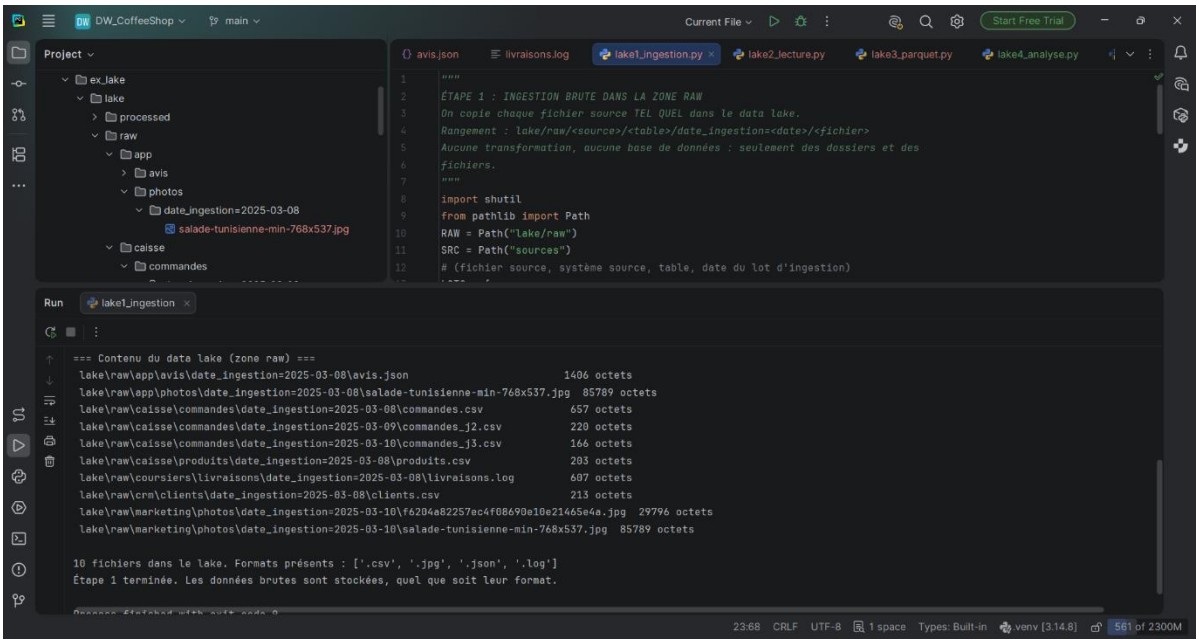
2. Un deuxième avis : la note de Walid devient 3



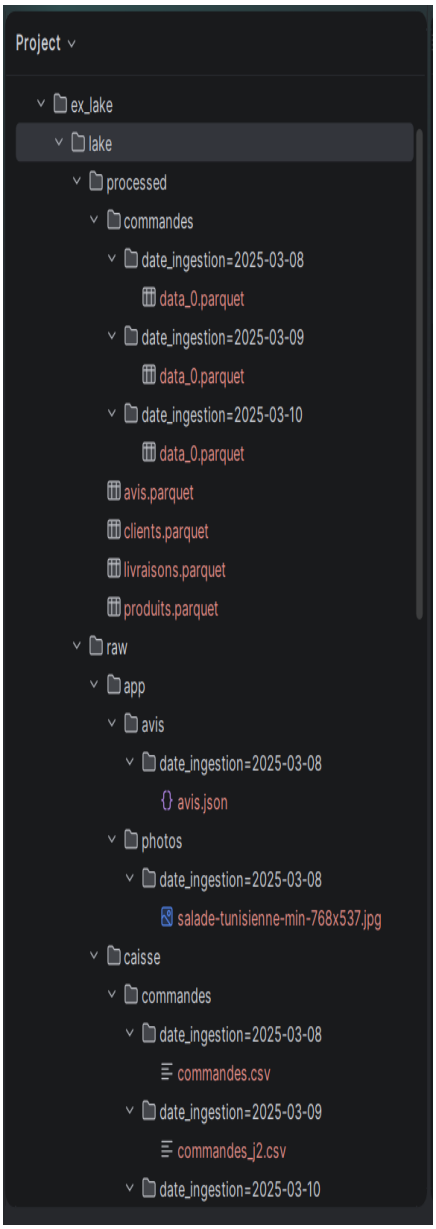
3. Une nouvelle colonne dans commande_j3.csv est acceptée à l'étape 2

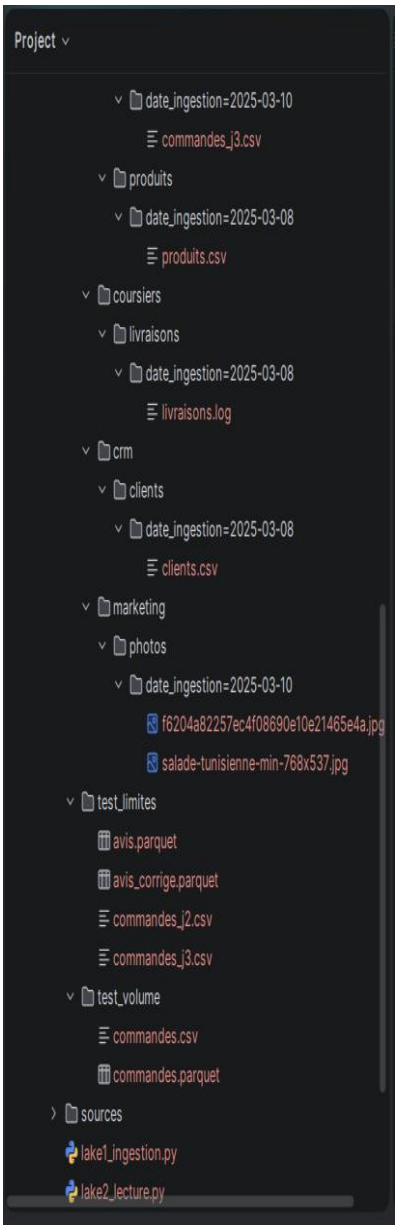


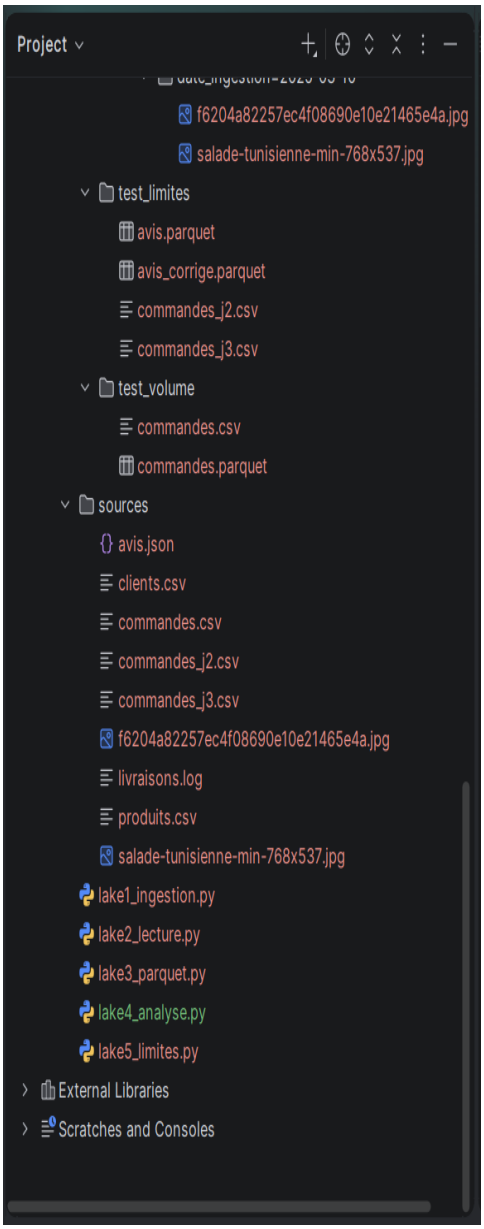
4. Un nouveau fichier image est accepté mais on ne peut pas l'interroger avec SQL.



5. Lake tree


Project
ex_lake
lake
processed
commandes
date_ingestion=2025-03-08
data_0.parquet
date_ingestion=2025-03-09
data_0.parquet
date_ingestion=2025-03-10
data_0.parquet
avis.parquet
clients.parquet
livraisons.parquet
produits.parquet
raw
app
avis
date_ingestion=2025-03-08
avis.json
photos
date_ingestion=2025-03-08
salade-tunisienne-min-768x537.jpg
caisse
commandes
date_ingestion=2025-03-08
commandes.csv
date_ingestion=2025-03-09
commandes_j2.csv
date_ingestion=2025-03-10


Project
date_ingestion=2025-03-10
commandes_j3.csv
produits
date_ingestion=2025-03-08
produits.csv
coursiers
livraisons
date_ingestion=2025-03-08
livraisons.log
crm
clients
date_ingestion=2025-03-08
clients.csv
marketing
photos
date_ingestion=2025-03-10
f6204a82257ec4f08690e10e21465e4a.jpg
salade-tunisienne-min-768x537.jpg
test_limites
avis.parquet
avis_corrige.parquet
commandes_j2.csv
commandes_j3.csv
test_volume
commandes.csv
commandes.parquet
sources
lake1_ingestion.py
lake2_lecture.py


Project
f6204a82257ec4f08690e10e21465e4a.jpg
salade-tunisienne-min-768x537.jpg
test_limites
avis.parquet
avis_corrige.parquet
commandes_j2.csv
commandes_j3.csv
test_volume
commandes.csv
commandes.parquet
sources
avis.json
clients.csv
commandes.csv
commandes_j2.csv
commandes_j3.csv
f6204a82257ec4f08690e10e21465e4a.jpg
livraisons.log
produits.csv
salade-tunisienne-min-768x537.jpg
lake1_ingestion.py
lake2_lecture.py
lake3_parquet.py
lake4_analyse.py
lake5_limites.py
External Libraries
Scratches and Consoles